

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1**



ANDROID BASIC WITH KOTLIN

Oleh:

ERISCHA MARSELA

NIM. 2410817120022

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basic With Kotlin Sederhana ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman II. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : ERISCHA MARSELA
NIM : 2410817120022

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom.
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI.....	3
DAFTAR TABEL	4
DAFTAR GAMBAR	5
SOAL 1	6
A. SOURCE CODE	7
B. OUTPUT CODE	20
C. PEMBAHASAN	23
SOAL 2	24
A. PEMBAHASAN	24
TAUTAN GIT.....	26
ADDITIONAL.....	26

DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt XML Soal 1.....	8
Tabel 2. Source Code Activity_main.xml Soal 1	9
Tabel 3. Source Code MainActivity.kt Compose Soal 1	13
Tabel 4. Source Code Testcase	28

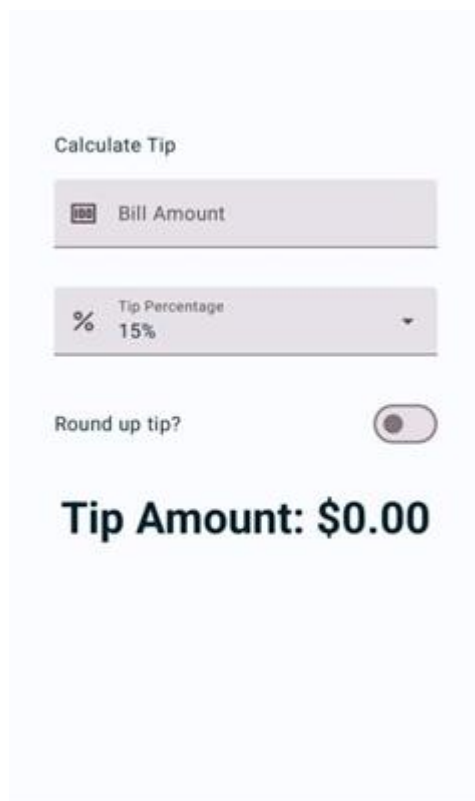
DAFTAR GAMBAR

Gambar 1. Tampilan Awalan Aplikasi.....	6
Gambar 2. Tampilan Pilihan Presentase Tip.....	7
Gambar 3. Tampilan Aplikasi Setelah Dijalankan.....	7
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML	20
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML	20
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML	21
Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose	21
Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose	22
Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose	22
Gambar 10. Output TestCase Extension Functions	29

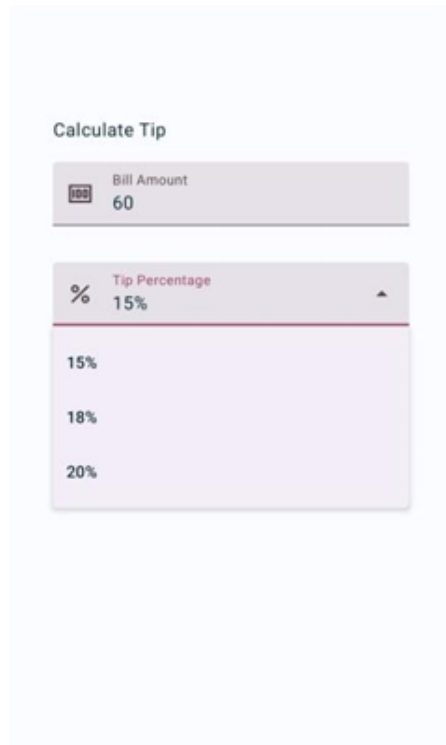
SOAL 1

Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

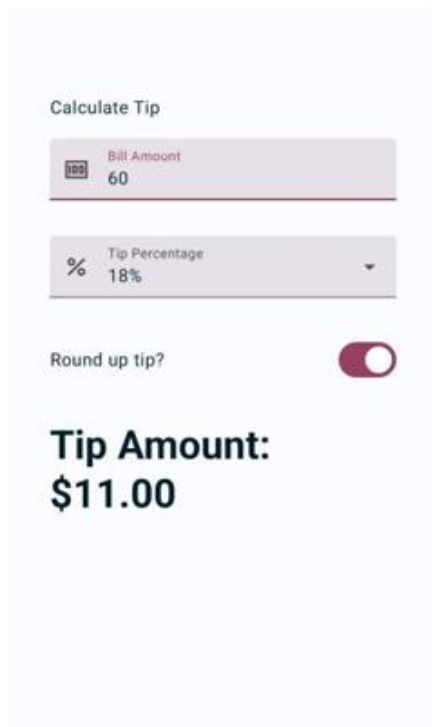
- Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
- Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
- Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
- Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1. Tampilan Awal Aplikasi



Gambar 2. Tampilan Pilihan Presentase Tip



Gambar 3. Tampilan Aplikasi Setelah Dijalankan

A. SOURCE CODE

MainActivity.Kt XML

Tabel 1. Source Code MainActivity.kt XML Soal 1

1.	package com.example.modul2xml
2.	
3.	import android.os.Bundle
4.	import android.widget.AdapterView
5.	import android.widget.Button
6.	import android.widget.EditText
7.	import android.widget.Spinner
8.	import android.widget.Switch
9.	import android.widget.TextView
10.	import androidx.appcompat.app.AppCompatActivity
11.	import kotlin.math.ceil
12.	
13.	class MainActivity : AppCompatActivity() {
14.	
15.	override fun onCreate(savedInstanceState: Bundle?) {
16.	super.onCreate(savedInstanceState)
17.	setContentView(R.layout.activity_main)
18.	
19.	val etTagihan = findViewById<EditText>(R.id.etTagihan)
20.	val spinnerTip = findViewById<Spinner>(R.id.spinnerTip)
21.	val switchBulat = findViewById<Switch>(R.id.switchBulat)
22.	val btnHitung = findViewById<Button>(R.id.btnHitung)
23.	val tvHasil = findViewById<TextView>(R.id.tvHasil)
24.	
25.	val pilihanPersentase = arrayOf("15%", "18%", "20%")

26.	val nilaiPersentase = arrayOf(0.15, 0.18, 0.20)
27.	
28.	val adapter = ArrayAdapter(this, android.R.layout.simple_spinner_dropdown_item, pilihanPersentase)
29.	spinnerTip.adapter = adapter
30.	
31.	btnHitung.setOnClickListener {
32.	val inputAngka = etTagihan.text.toString()
33.	val tagihan = inputAngka.toDoubleOrNull() ?: 0.0
34.	
35.	val posisiTerpilih = spinnerTip.selectedItemPosition
36.	
37.	val persentaseTip = nilaiPersentase[posisiTerpilih]
38.	
39.	var jumlahTip = tagihan * persentaseTip
40.	
41.	if (switchBulat.isChecked) {
42.	jumlahTip = ceil(jumlahTip)
43.	}
44.	
45.	tvHasil.text = String.format("Jumlah Tip: \$%.2f", jumlahTip)
46.	}
47.	}
48.	}

Activity_main.xml

Tabel 2. Source Code Activity_main.xml Soal 1

1.	<?xml version="1.0" encoding="utf-8"?>
2.	<LinearLayout
3.	xmlns:android="http://schemas.android.com/apk/res/android"
4.	android:layout_width="match_parent"
5.	android:layout_height="match_parent"
6.	android:orientation="vertical"
7.	android:gravity="center_vertical"
8.	android:padding="32dp">
9.	<TextView
10.	android:layout_width="wrap_content"
11.	android:layout_height="wrap_content"
12.	android:text="Calculate Tip"
13.	android:textSize="20sp"
14.	android:textStyle="bold"
15.	android:layout_marginBottom="24dp"/>
16.	
17.	<EditText
18.	android:id="@+id/etTagihan"
19.	android:layout_width="match_parent"
20.	android:layout_height="60dp"
21.	android:hint="Bill Amount"
22.	android:inputType="numberDecimal"
23.	android:background="#F3F3F6"
24.	android:paddingStart="16dp"
25.	android:paddingEnd="16dp"
26.	android:layout_marginBottom="24dp" />
27.	
28.	<LinearLayout
29.	android:layout_width="match_parent"

30.	android:layout_height="60dp"
31.	android:orientation="horizontal"
32.	android:background="#F3F3F6"
33.	android:gravity="center_vertical"
34.	android:layout_marginBottom="32dp">
35.	
36.	<TextView
37.	android:layout_width="wrap_content"
38.	android:layout_height="wrap_content"
39.	android:text="%"
40.	android:textSize="18sp"
41.	android:textStyle="bold"
42.	android:paddingStart="16dp"
43.	android:paddingEnd="16dp"/>
44.	
45.	<Spinner
46.	android:id="@+id/spinnerTip"
47.	android:layout_width="match_parent"
48.	android:layout_height="match_parent"
49.	android:background="@null" />
50.	</LinearLayout>
51.	
52.	<LinearLayout
53.	android:layout_width="match_parent"
54.	android:layout_height="wrap_content"
55.	android:orientation="horizontal"
56.	android:gravity="center_vertical"
57.	android:layout_marginBottom="32dp">

58.	
59.	<TextView
60.	android:layout_width="0dp"
61.	android:layout_height="wrap_content"
62.	android:layout_weight="1"
63.	android:text="Round up tip?"
64.	android:textSize="16sp"/>
65.	
66.	<Switch
67.	android:id="@+id/switchBulat"
68.	android:layout_width="wrap_content"
69.	android:layout_height="wrap_content" />
70.	</LinearLayout>
71.	
72.	<Button
73.	android:id="@+id/btnHitung"
74.	android:layout_width="match_parent"
75.	android:layout_height="wrap_content"
76.	android:text="Tip Amount"/>
77.	
78.	<TextView
79.	android:id="@+id/tvHasil"
80.	android:layout_width="wrap_content"
81.	android:layout_height="wrap_content"
82.	android:text="Tip Amount: \$0.00"
83.	android:textSize="32sp"
84.	android:textStyle="bold"
85.	android:layout_marginTop="16dp"/>
86.	

87.	</LinearLayout>
-----	-----------------

MainActivity.Kt Compose

Tabel 3. Source Code MainActivity.kt Compose Soal 1

1.	package com.example.modul2compose
2.	
3.	import android.os.Bundle
4.	import androidx.activity.ComponentActivity
5.	import androidx.activity.compose.setContent
6.	import androidx.compose.foundation.layout.*
7.	import androidx.compose.foundation.text.KeyboardOptions
8.	import androidx.compose.material3.*
9.	import androidx.compose.runtime.*
10.	import androidx.compose.ui.Alignment
11.	import androidx.compose.ui.Modifier
12.	import androidx.compose.ui.graphics.Color
13.	import androidx.compose.ui.text.font.FontWeight
14.	import androidx.compose.ui.text.input.KeyboardType
15.	import androidx.compose.ui.unit.dp
16.	import androidx.compose.ui.unit.sp
17.	import kotlin.math.ceil
18.	
19.	class MainActivity : ComponentActivity() {
20.	override fun onCreate(savedInstanceState: Bundle?) {
21.	super.onCreate(savedInstanceState)
22.	setContent {
23.	MaterialTheme {
24.	Surface(
25.	modifier Modifier.fillMaxSize(),
	=

```

26.         color =
MaterialTheme.colorScheme.background
27.     ) {
28.         KalkulatorTipCompose()
29.     }
30. }
31. }
32. }
33. }
34.
35. @OptIn(ExperimentalMaterial3Api::class)
36. @Composable
37. fun KalkulatorTipCompose() {
38.     var inputTagihan by remember {
mutableStateOf("") }
39.
40.     var dropdownExpanded by remember {
mutableStateOf(false) }
41.     val daftarPilihan = listOf("15%" to 0.15,
"18%" to 0.18, "20%" to 0.20)
42.     var pilihanTerpilih by remember {
mutableStateOf(daftarPilihan[0]) }
43.
44.     var switchBulat by remember {
mutableStateOf(false) }
45.     var teksHasil by remember {
mutableStateOf("Tip Amount: $0.00") }
46.
47.     Column(
48.         modifier = Modifier
49.             .fillMaxSize()
50.             .padding(32.dp),
51.         verticalArrangement = Arrangement.Center

```

```

52.         ) {
53.
54.             Text(
55.                 text = "Calculate Tip",
56.                 fontSize = 20.sp,
57.                 fontWeight = FontWeight.Bold,
58.                 modifier = Modifier.padding(bottom =
24.dp)
59.             )
60.
61.             TextField(
62.                 value = inputTagihan,
63.                 onChange = { inputTagihan = it },
64.                 placeholder = { Text( text = "Bill
Amount", fontSize = 18.sp,) },
65.                 keyboardOptions =
KeyboardOptions(keyboardType =
KeyboardType.Number),
66.                 modifier = Modifier
67.                     .fillMaxWidth()
68.                     .height(60.dp)
69.                     .padding(bottom = 8.dp),
70.                 leadingIcon = {
71.                     Text(
72.                         text = "$",
73.                         fontSize = 18.sp,
74.                         fontWeight = FontWeight.Bold,
75.                         modifier =
Modifier.padding(start = 16.dp, end = 8.dp)
76.                     )
77.                 },
78.                 colors = TextFieldDefaults.colors(

```

```

79.             unfocusedContainerColor           =
Color(0xFFFF3F3F6),
80.             focusedContainerColor             =
Color(0xFFFF3F3F6),
81.             unfocusedIndicatorColor           =
Color.Transparent,
82.             focusedIndicatorColor             =
Color.Transparent
83.         )
84.     )
85.
86.     ExposedDropDownMenuBox(
87.         expanded = dropdownExpanded,
88.         onExpandedChange = { dropdownExpanded
= !dropdownExpanded },
89.         modifier = Modifier.padding(bottom =
32.dp)
90.     ) {
91.         TextField(
92.             value = pilihanTerpilih.first,
93.             onValueChange = {},
94.             readOnly = true,
95.             leadingIcon = {
96.                 Text(
97.                     text = "%",
98.                     fontSize = 18.sp,
99.                     fontWeight           =
FontWeight.Bold,
100.                    modifier           =
Modifier.padding(start = 16.dp, end = 8.dp)
101.                )
102.            },

```

```

103.         trailingIcon = {
ExposedDropDownMenuDefaults.TrailingIcon(expanded
= dropdownExpanded) },
104.         modifier = Modifier
105.             .menuAnchor()
106.             .fillMaxWidth()
107.             .height(60.dp),
108.         colors =
TextFieldDefaults.colors(
109.             unfocusedContainerColor =
Color(0xFFFF3F3F6),
110.             focusedContainerColor =
Color(0xFFFF3F3F6),
111.             unfocusedIndicatorColor =
Color.Transparent,
112.             focusedIndicatorColor =
Color.Transparent
113.         )
114.     )
115.     ExposedDropDownMenu(
116.         expanded = dropdownExpanded,
117.         onDismissRequest = {
dropdownExpanded = false }
118.     ) {
119.         daftarPilihan.forEach { pilihan -
>
120.             DropdownMenuItem(
121.                 text = {
Text(pilihan.first) },
122.                 onClick = {
123.                     pilihanTerpilih =
pilihan
124.                     dropdownExpanded =
false
125.                 }

```

```

126.         )
127.     }
128. }
129. }
130.
131. Row(
132.     modifier = Modifier
133.         .fillMaxWidth()
134.         .padding(bottom = 32.dp),
135.     verticalAlignment =
Alignment.CenterVertically
136. ) {
137.     Text(
138.         text = "Round up tip?",
139.         fontSize = 16.sp,
140.         modifier = Modifier.weight(1f)
141.     )
142.     Switch(
143.         checked = switchBulat,
144.         onCheckedChange = { switchBulat =
it }
145.     )
146. }
147.
148. Button(
149.     onClick = {
150.         val tagihan =
inputTagihan.toDoubleOrNull() ?: 0.0
151.
152.         val persentaseTip =
pilihanTerpilih.second
153.

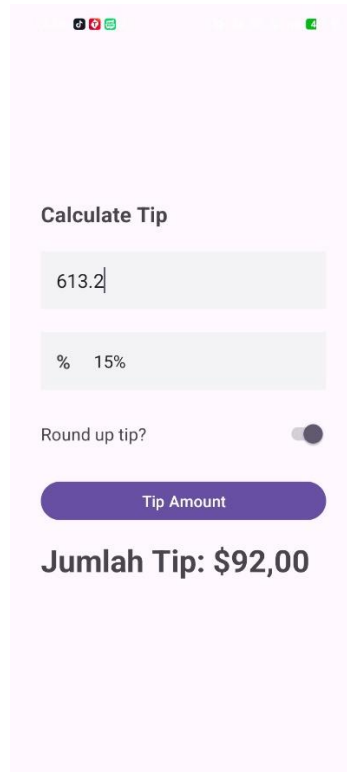
```

```

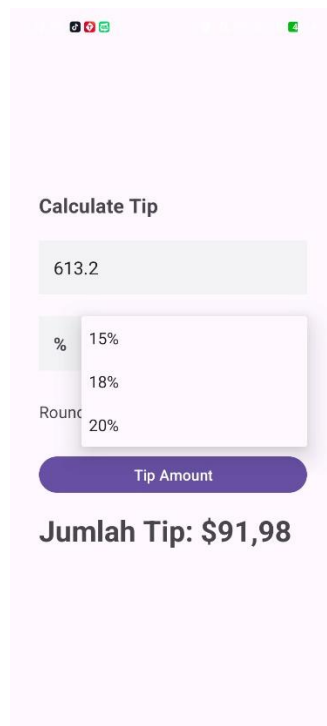
154.         var    jumlahTip    =    tagihan    *
persentaseTip
155.
156.         if (switchBulat) {
157.             jumlahTip = ceil(jumlahTip)
158.         }
159.
160.         teksHasil    =    String.format("Tip
Amount: $%.2f", jumlahTip)
161.         },
162.         modifier = Modifier.fillMaxWidth()
163.     ) {
164.         Text("Tip Amount")
165.     }
166.
167.     Text(
168.         text = teksHasil,
169.         fontSize = 32.sp,
170.         fontWeight = FontWeight.Bold,
171.         modifier    =    Modifier.padding(top    =
16.dp)
172.     )
173. }
174. }

```

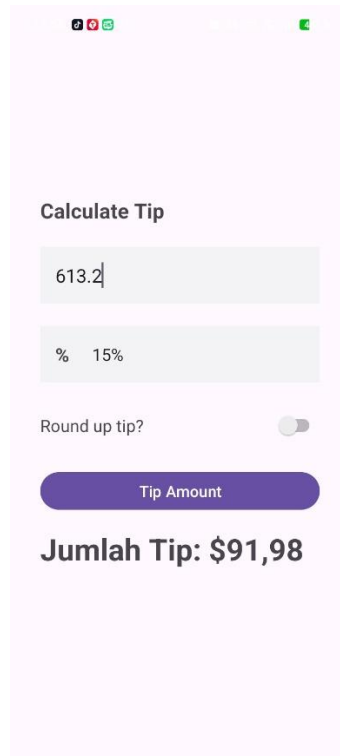
B. OUTPUT CODE



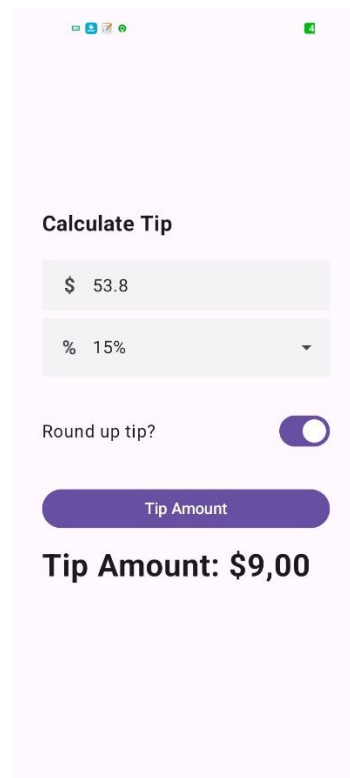
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML



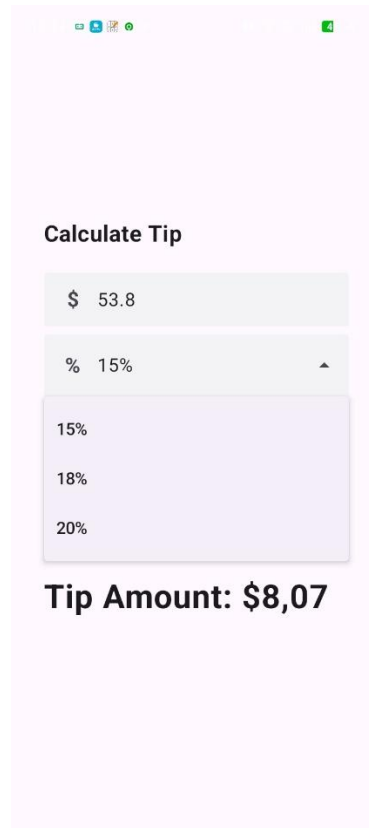
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML



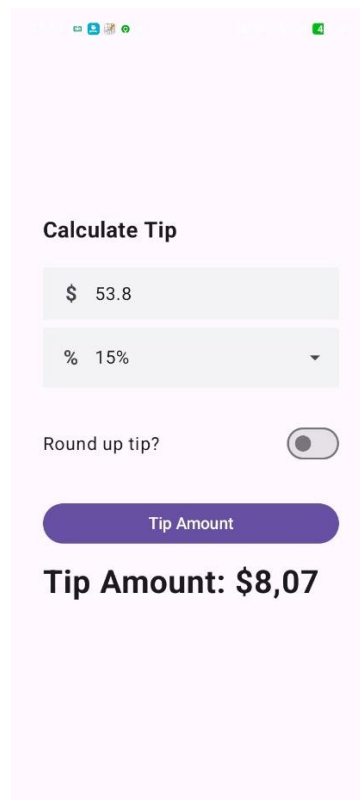
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML



Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose

C. PEMBAHASAN

MainActivity.ky XML

“`SetContentView`” untuk menampilkan tata letak statis dari file `activity_main.xml` ke layar. Selanjutnya, fungsi “`findViewById`” digunakan untuk mencari dan menghubungkan komponen desain seperti kotak input, menu tarik-turun, dan sakelar agar dapat dimanipulasi melalui kode Kotlin. Setelah komponen terhubung, program mendeklarasikan variabel array untuk menyimpan teks persentase tip beserta nilai desimalnya. Data teks tersebut kemudian dipetakan ke dalam komponen menu tarik-turun (Spinner) dengan bantuan “`ArrayAdapter`”. Bagian inti dari logika ini berada di dalam *event listener* pada tombol hitung (`btnHitung.setOnClickListener`), yang akan mengeksekusi perhitungan hanya ketika tombol ditekan. Di dalam blok tersebut, program mengambil teks input dan mengonversinya menjadi angka desimal menggunakan “`toDoubleOrNull`”, mengambil nilai persentase berdasarkan posisi yang dipilih pengguna, lalu menghitung total tip. Jika opsi pembulatan pada sakelar diaktifkan, fungsi “`ceil`” akan membulatkan hasil tersebut ke bilangan bulat di atasnya sebelum akhirnya diformat menjadi teks mata uang dan ditampilkan ke layar.

Activity_main.xml

`activity_main.xml` berfungsi murni sebagai cetak biru visual aplikasi yang ditulis menggunakan bahasa penanda XML. Tampilan aplikasi ini menggunakan “`LinearLayout`” dengan orientasi vertikal sebagai wadah utama untuk memastikan semua elemen antarmuka disusun rapi secara bertumpuk dari atas ke bawah. Di dalam wadah tersebut, terdapat berbagai komponen dasar seperti “`TextView`” yang bertugas menampilkan teks statis untuk judul dan label, serta “`EditText`” dengan tipe input angka desimal agar pengguna hanya bisa mengetikkan nominal tagihan. Untuk memberikan opsi persentase, kode ini memanfaatkan komponen “`Spinner`” yang menghasilkan menu dropdown. Selain itu, ditambahkan pula komponen “`Switch`” yang bertindak sebagai sakelar interaktif (On/Off) untuk fitur pembulatan nilai tip. Susunan antarmuka ini kemudian ditutup dengan sebuah “`Button`” yang berfungsi sebagai pemicu agar file Kotlin memulai proses komputasi ketika tombol tersebut ditekan.

MainActivity.kt Compose

“setContent” yang menandakan bahwa tampilan akan digambar sepenuhnya oleh mesin Compose. Konsep paling krusial dalam kode ini adalah penggunaan State melalui fungsi “remember { mutableStateOf(...) }”. Variabel state ini sangat reaktif; jika nilainya berubah (misalnya saat pengguna mengetik angka atau hasil dihitung), bagian layar yang terkait dengan variabel tersebut akan otomatis digambar ulang. Tata letaknya sendiri dibangun secara intuitif menggunakan komponen Column untuk menyusun elemen secara vertikal dan “Row” untuk susunan horizontal. Perubahan komponen juga sangat terlihat; kotak input kini menggunakan “TextField”, sedangkan menu dropdown dibangun menggunakan kombinasi “ExposedDropdownMenuBox” dan perulangan baris data. Seluruh interaksi diakhiri pada fungsi “onClick” di dalam “Button”, di mana program langsung mengeksekusi perhitungan matematis dari nilai state yang ada, membulatkannya jika sakelar aktif, dan langsung memperbarui state hasil sehingga teks jumlah tip di layar otomatis berubah.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. PEMBAHASAN

Perbedaan mendasar antara implementasi XML dan Jetpack Compose terletak pada paradigma pemrograman dan pengelolaan tampilannya. Pendekatan XML menggunakan paradigma imperatif di mana desain antarmuka (UI) dipisahkan secara fisik di dalam file berformat .xml, sementara logika program ditulis di file Kotlin atau Java, sehingga pengembang harus secara manual menuliskan perintah untuk memperbarui elemen UI setiap kali ada perubahan data (misalnya memanggil `textView.text = "Baru"`). Sebaliknya, Jetpack Compose menggunakan paradigma deklaratif yang menyatukan pembuatan desain antarmuka dan logika ke dalam satu bahasa, yaitu Kotlin, melalui fungsi beranotasi `@Composable`. Alih-alih memerintahkan "bagaimana" UI harus diubah, Compose memungkinkan pengembang mendeklarasikan "apa" yang harus ditampilkan

berdasarkan status (state) data saat ini, dan UI akan secara otomatis menggambar ulang (recompose) dirinya sendiri ketika state tersebut mengalami perubahan.

Implementasi menggunakan XML memiliki kelebihan utama berupa ekosistem yang sudah sangat matang dan stabil, mengingat metode ini telah menjadi standar pengembangan Android selama belasan tahun, sehingga dokumentasi, referensi perbaikan bug, serta dukungan pustaka (library) pihak ketiga sangat melimpah. Selain itu, XML memberikan pemisahan peran yang tegas antara desain visual dan logika bisnis (separation of concerns), yang didukung oleh alat Layout Editor bawaan Android Studio untuk melakukan pratinjau dan desain drag and drop dengan sangat ringan. Namun, kelemahan utama XML adalah banyaknya boilerplate atau kode panjang yang berulang, khususnya saat membangun daftar bergulir yang rumit (seperti keharusan membuat Adapter dan ViewHolder pada RecyclerView). Selain itu, karena pembaruan UI dilakukan secara manual, metode ini sangat rentan terhadap bug sinkronisasi, di mana data di latar belakang sudah berubah namun tampilan di layar gagal diperbarui karena pengembang terlewat menambahkan baris kode pembaruannya.

Di sisi lain, kelebihan utama Jetpack Compose adalah penulisan kode yang jauh lebih ringkas, intuitif, dan efisien sehingga secara drastis memangkas kode boilerplate yang sebelumnya wajib ada di XML. Penggunaan satu bahasa (Kotlin) untuk membangun antarmuka dan logika membuat pengembangan menjadi lebih terpadu, ditambah dengan kemampuannya yang sangat reaktif sehingga antarmuka selalu terjamin selaras dengan kondisi data (state) terbaru tanpa perlu pembaruan manual. Meski menawarkan banyak kemudahan, kekurangan dari Jetpack Compose adalah kurva pembelajarannya yang cukup curam, terutama bagi pengembang yang sudah terbiasa memanipulasi UI secara imperatif, karena mereka harus merombak pola pikir menjadi reaktif. Selain itu, fitur pratinjau (Preview) Compose di Android Studio sering kali membutuhkan lebih banyak alokasi memori (RAM) serta waktu build yang lebih lama dibandingkan Layout Editor XML, dan masih ada beberapa pustaka lama yang belum mendukung Compose secara penuh tanpa trik penyesuaian (interoperability).

TAUTAN GIT

<https://github.com/ERISCHA24/PRAKTIKUMMOBILE.git>

ADDITIONAL

1. Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**

a) Null safety

Dalam lingkungan Java, NullPointerException (NPE) merupakan anomali yang sering memicu kegagalan sistem pada saat runtime. Kotlin mereduksi risiko ini secara fundamental melalui implementasi Null Safety pada tingkat sistem tipe (type system). Kompiler Kotlin membedakan secara ketat antara referensi yang diizinkan untuk menampung nilai kosong (nullable references) dan yang tidak diizinkan (non-nullable references). Mekanisme ini memaksa pengembang untuk menangani potensi nilai null pada fase kompilasi (compile-time), sehingga meningkatkan stabilitas dan keandalan perangkat lunak secara signifikan sebelum aplikasi dijalankan.

b) Chaining

Method Chaining merupakan paradigma sintaksis yang memfasilitasi pemanggilan serangkaian fungsi secara sekuensial dalam satu instruksi tunggal. Pada Kotlin, fitur ini dioptimalkan melalui implementasi scope functions (seperti “let”, “run”, “with”, “apply”, dan “also”). Hal ini memungkinkan setiap pemanggilan metode untuk memproses sebuah objek dan mengembalikannya, yang kemudian langsung menjadi subjek bagi eksekusi metode selanjutnya. Pendekatan ini secara efektif meningkatkan tingkat keterbacaan kode (code readability) dan mereduksi kompleksitas siklomatik dengan meminimalisasi deklarasi variabel sementara yang berlebihan.

c) Extension Functions

Fungsi ekstensi merupakan mekanisme yang mengizinkan pengembang untuk mengaugmentasi fungsi perilaku baru ke dalam suatu kelas yang telah ada tanpa perlu mengimplementasikan pewarisan hierarkis (inheritance) atau mengadopsi pola desain dekorator (decorator design pattern). Fitur ini memungkinkan modifikasi pada kelas-kelas tertutup (final classes), seperti komponen antarmuka pengguna (View) atau kelas dasar (String). Secara leksikal, fungsi ekstensi dipanggil seolah-olah merupakan metode internal dari kelas tersebut, yang pada gilirannya menghasilkan struktur kode yang lebih terenkapsulasi dan modular.

d) Data class

Dalam paradigma berorientasi objek konvensional, pendefinisian entitas data murni (Plain Old Java Object / POJO) mewajibkan penulisan kode repetitif (boilerplate code) yang substansial, meliputi deklarasi fungsi aksesori (getter dan setter), “equals()”, “hashCode()”, dan “toString()”. Kotlin mengatasi inefisiensi ini melalui sintaksis “data class”. Kompiler secara otomatis mensintesis seluruh fungsi standar yang diperlukan untuk representasi dan manipulasi data pada level sistem, sehingga menghasilkan arsitektur kode yang lebih ringkas serta meminimalisasi potensi human error dalam pendefinisian entitas data.

Sumber =

* **Jamern, Dmitry & Isakova, Svetlana (2017).** Kotlin in Action. ([Books/Kotlin in Action.Dmitry Jemerov Svetlana Isakova .pdf at master · IlyaShelkovenko/Books](#))

* **Subramaniam, Venkat (2019).** Programming Kotlin: Create Elegant, Expressive, and Performant JVM and Android Applications. ([Download PDF - Programming Kotlin: Create Elegant, Expressive, And Performant Jvm And Android Applications \[PDF\] \[sf7deeissko0\]](#))

2. berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

Kita akan membuat fungsi tambahan untuk kelas bawaan Kotlin, yaitu Int (Integer/Angka Bawah). Kita akan menambahkan fungsi baru bernama `formatRupiah()` agar setiap angka bisa langsung mengubah dirinya menjadi format teks mata uang tanpa perlu membuat kelas bantuan (utility class) yang rumit.

“`fun Int.formatRupiah()`” ini memberi tahu Kotlin, "Tambahkan fungsi `formatRupiah` ke dalam kelas `Int`". Padahal, kelas `Int` adalah kelas bawaan inti dari Kotlin yang source codenya tidak bisa kita ubah. Di dalam extension function, “`this`” mewakili objek yang memanggilnya. Jika “`150000.formatRupiah()`” dipanggil, maka “`this`” bernilai 150000.

Tabel 4. Source Code Testcase

1.	<code>fun Int.formatRupiah(): String {</code>
2.	<code> return "Rp \$this,00"</code>
3.	<code>}</code>
4.	
5.	<code>fun main() {</code>
6.	<code> val hargaBaju = 150000</code>
7.	<code> val hargaSepatu = 350000</code>
8.	
9.	<code> val teksBaju = hargaBaju.formatRupiah()</code>
10.	<code> val teksSepatu = hargaSepatu.formatRupiah()</code>
11.	
12.	<code> println("Harga Baju : \$teksBaju")</code>
13.	<code> println("Harga Sepatu : \$teksSepatu")</code>
14.	<code> println("Harga Topi : " +</code> <code>45000.formatRupiah())</code>
15.	<code>}</code>

```
"C:\Program Files\Java\jdk-25\bin\java.  
Harga Baju    : Rp 150000,00  
Harga Sepatu  : Rp 350000,00  
Harga Topi    : Rp 45000,00  
  
Process finished with exit code 0
```

Gambar 10. Output TestCase Extension Functions